
LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK



NAMA : MUHAMAD KRISNA YODI PRATAMA

NIM : 251104410083

PERIODE: SEMESTER GENAP 2025/2026

**PROGRAM STUDI TEKNIK INFORMATIKA FAKULTAS TEKNOLOGI
INFORMASI UNIVERSITAS ISLAM BALITAR 2026**

BAB I

ANALISA SOURCE CODE (MODUL 7.7)

Sesuai dengan instruksi pada Modul 7.7, berikut adalah analisa dari setiap source code dan studi kasus yang telah dijelaskan pada modul, termasuk penyelesaian untuk pertanyaan analisa pada sub-bab 7.3 dan 7.5.

1.1 Analisa Penambahan Method dan Overriding (Sub-bab 7.3)

Pertanyaan Analisa: *Tambahkan method setMesin dan getMesin di class MainKendaraan seperti coding pada gambar 7.3. Run kembali, bagaimana hasilnya! Jelaskan!*

Source Code Analisa:

```
class Kendaraan {
    protected String mesin;
    public void info() {
        System.out.println("Ini adalah Kendaraan");
    }
}

class MainKendaraan extends Kendaraan {
    // Menambahkan setter dan getter di subclass
    public void setMesin(String mesin) {
        this.mesin = mesin; // Mengakses atribut protected dari parent
    }
    public String getMesin() {
        return mesin;
    }

    @Override
    public void info() {
        System.out.println("Ini adalah MainKendaraan dengan mesin: " +
getMesin());
    }
}

public class TestKendaraan {
    public static void main(String[] args) {
        // Virtual Method Invocation (VMI)
        Kendaraan k = new MainKendaraan();
        // k.setMesin("V8"); // Error jika tidak di-casting, karena referensi
Kendaraan tidak mengenal method setMesin
        ((MainKendaraan) k).setMesin("V8 Turbo"); // Casting diperlukan
    }
}
```

```

        k.info();
    }
}

```

Hasil Output:

Ini adalah MainKendaraan dengan mesin: V8 Turbo

[Masukkan Screenshot Output dari IDE Anda di sini]

Penjelasan Analisa: Ketika method `setMesin` dan `getMesin` ditambahkan ke dalam `MainKendaraan` (child class), subclass tersebut dapat memodifikasi atribut `mesin` yang diwarisi dari parent class karena atribut tersebut memiliki modifier `protected`. Namun, jika kita menggunakan konsep *Virtual Method Invocation* (VMI) di mana objek parent mereferensikan child (`Kendaraan k = new MainKendaraan()`), kita tidak bisa langsung memanggil `setMesin()` tanpa melakukan *casting*. Hal ini karena compiler membaca referensi tipe `Kendaraan` yang tidak memiliki method `setMesin`. Ketika method `info()` dipanggil, terjadi *overriding*, sehingga compiler menjalankan method `info()` milik `MainKendaraan` (child), yang kemudian menampilkan nilai mesin yang telah di-set.

1.2 Analisa Single Inheritance dan Modifier Private (Sub-bab 7.5)

Pertanyaan Analisa 1: *Bagaimana hasilnya jika pada class Mamalia, modifier variable nama diubah menjadi private? jelaskan!*

Penjelasan Analisa: Jika modifier variabel `nama` pada class `Mamalia` (parent) diubah menjadi `private`, maka class `Kucing` (child) **tidak akan bisa mengakses** variabel `nama` tersebut secara langsung. Dalam konsep *Inheritance*, member dengan hak akses `private` tidak diturunkan/diwariskan ke subclass. Jika pada class `Kucing` terdapat kode yang mencoba memanggil atau memodifikasi `nama` secara langsung (misal: `this.nama = "Tom"`), maka akan terjadi **Compile-Time Error** (error saat kompilasi). Untuk mengatasinya, class `Mamalia` harus menyediakan method *getter* dan *setter* dengan modifier `public` atau `protected`.

Pertanyaan Analisa 2: *Cobalah melakukan VMI pada object cat, dan lakukan override untuk method speak sehingga menjalankan method speak tersebut dari parent-nya.*

Source Code Analisa:

```

class Mamalia {
    protected String nama;
    public Mamalia(String nama) { this.nama = nama; }
    public void speak() {
        System.out.println("Suara generic dari Mamalia");
    }
}

```

```

class Kucing extends Mamalia {
    public Kucing(String nama) { super(nama); }

    @Override
    public void speak() {
        super.speak(); // Memanggil method speak dari parent-nya
        System.out.println("Meong! Nama saya " + nama);
    }
}

public class TestMamalia {
    public static void main(String[] args) {
        // Virtual Method Invocation (VMI)
        Mamalia cat = new Kucing("Tom");
        cat.speak();
    }
}

```

Hasil Output:

```

Suara generic dari Mamalia
Meong! Nama saya Tom

```

[Masukkan Screenshot Output dari IDE Anda di sini]

Penjelasan Analisa: Pada source code di atas, diterapkan konsep VMI (`Mamalia cat = new Kucing("Tom")`). Saat `cat.speak()` dipanggil, Java akan mengeksekusi method `speak()` yang sudah di-*override* di class `Kucing`. Untuk memenuhi instruksi "menjalankan method speak dari parent-nya", digunakan keyword `super.speak()` di dalam method `speak()` milik `Kucing`. Keyword `super` digunakan untuk mengakses method atau konstruktor dari class induk (parent class).

BAB II

TUGAS PRAKTIKUM (MODUL 7.8)

2.1 Design Aplikasi Toko Buku

Sesuai dengan ketentuan tugas, aplikasi toko buku akan didesain menggunakan konsep *Inheritance*.

- **Parent Class (Superclass):** Buku
 - Berisi atribut umum yang pasti dimiliki oleh semua jenis buku: `judul`, `nama_pengarang`, `penerbit`, dan `tahun_cetak`.

- **Child Class (Subclass):** `BukuToko`
 - Mewarisi atribut dari class `Buku` dan menambahkan atribut spesifik untuk keperluan toko, yaitu `kategori`. Class ini juga akan memiliki method untuk memproses kode kategori dan menampilkan informasi (meng-*override* method `tampil`).
- **Main Class:** `MainTokoBuku`
 - Berisi logika aplikasi untuk entri data (menggunakan `Scanner`) dan menampilkan data buku yang telah diinput.

2.2 Implementasi Source Code (Java)

1. Parent Class: `Buku.java`

```
package maintokobuku;

public class Buku {
    // Atribut parent class
    protected String judul;
    protected String nama_pengarang;
    protected String penerbit;
    protected int tahun_cetak;

    // Constructor
    public Buku(String judul, String nama_pengarang, String penerbit, int tahun_cetak) {
        this.judul = judul;
        this.nama_pengarang = nama_pengarang;
        this.penerbit = penerbit;
        this.tahun_cetak = tahun_cetak;
    }

    // Method untuk menampilkan informasi
    public void displayInfo() {
        System.out.println("Judul Buku      : " + judul);
        System.out.println("Pengarang      : " + nama_pengarang);
        System.out.println("Penerbit       : " + penerbit);
        System.out.println("Tahun Cetak    : " + tahun_cetak);
    }
}
```

2. Child Class: `BukuToko.java`

```

class BukuToko extends Buku {
    // 1. Ditambahkan 'final' karena nilainya tidak diubah setelah constructor
    private final String kategori;

    public BukuToko(String judul, String nama_pengarang, String penerbit, int tahun_cetak, String kategori) {
        super(judul, nama_pengarang, penerbit, tahun_cetak);
        this.kategori = kategori;
    }

    // 2. Menggunakan Switch Expression (Java Modern) dengan tanda panah (->)
    public String getDeskripsiKategori() {
        // Pencegahan error jika kategori kosong (null)
        if (kategori == null) {
            return "Kategori Tidak Dikenal";
        }

        // Switch Expression (Lebih singkat dan rapi)
        return switch (kategori.toLowerCase()) {
            case "su" -> "Semua Umur";
            case "r" -> "Remaja";
            case "d" -> "Dewasa";
            case "a" -> "Anak";
            default -> "Kategori Tidak Dikenal";
        };
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Kategori : " + getDeskripsiKategori());
        System.out.println("-----");
    }
}

```

3. Main Class: MainTokoBuku.java

```

// 3. MAIN CLASS (Hanya ini yang boleh 'public')
public class MainTokoBuku {
    public static void main(String[] args) {

        // Menggunakan Try-With-Resources (Scanner diletakkan di dalam kurung try)
        try (java.util.Scanner scanner = new java.util.Scanner(System.in)) {

            BukuToko[] daftarBuku = new BukuToko[3];

            System.out.println("=== APLIKASI ENTRI DATA TOKO BUKU ===");

            // Proses Input Data Sebanyak 3 Kali
            for (int i = 0; i < 3; i++) {
                System.out.println("\n--- Input Data Buku ke-" + (i + 1) + " ---");

                System.out.print("Judul Buku      : ");
                String judul = scanner.nextLine();

                System.out.print("Nama Pengarang : ");
                String pengarang = scanner.nextLine();

                System.out.print("Penerbit      : ");
                String penerbit = scanner.nextLine();

                System.out.print("Tahun Cetak   : ");
                int tahun = Integer.parseInt(scanner.nextLine());

                System.out.print("Kategori (su/r/d/a) : ");
                String kat = scanner.nextLine();

                // Instansiasi objek child class
                daftarBuku[i] = new BukuToko(judul, pengarang, penerbit, tahun, kat);
            }

            // Proses Menampilkan Isi Buku
            System.out.println("\n\n=== DAFTAR ISI BUKU TOKO ===");
            System.out.println("=====");
            for (int i = 0; i < 3; i++) {
                System.out.println("Data Buku ke-" + (i + 1));
                daftarBuku[i].displayInfo();
            }

        } catch (Exception e) {
            // Menangkap error jika terjadi kesalahan input (misal: tahun cetak diisi huruf)
            System.out.println("Terjadi kesalahan input: " + e.getMessage());
        }

        // TIDAK PERLU lagi menulis scanner.close(); di sini, karena sudah ditutup otomatis oleh try()
    }
}

```

2.3 Hasil Output Program

Berikut adalah simulasi hasil running program ketika pengguna memasukkan 3 data buku dengan kategori yang berbeda-beda.

```
=== APLIKASI ENTRI DATA TOKO BUKU ===
```

```

--- Input Data Buku ke-1 ---
Judul Buku      : Laskar Pelangi
Nama Pengarang  : Andrea Hirata
Penerbit        : Bentang Pustaka
Tahun Cetak     : 2005
Kategori (su/r/d/a) : su

```

--- Input Data Buku ke-2 ---

Judul Buku : Bumi Manusia
Nama Pengarang : Pramoedya Ananta Toer
Penerbit : Hasta Mitra
Tahun Cetak : 1980
Kategori (su/r/d/a) : d

--- Input Data Buku ke-3 ---

Judul Buku : Narnia
Nama Pengarang : C.S. Lewis
Penerbit : Gramedia
Tahun Cetak : 2004
Kategori (su/r/d/a) : a

=== DAFTAR ISI BUKU TOKO ===

=====

Data Buku ke-1

Judul Buku : Laskar Pelangi
Pengarang : Andrea Hirata
Penerbit : Bentang Pustaka
Tahun Cetak : 2005
Kategori : Semua Umur

=====

Data Buku ke-2

Judul Buku : Bumi Manusia
Pengarang : Pramoedya Ananta Toer
Penerbit : Hasta Mitra
Tahun Cetak : 1980
Kategori : Dewasa

=====

Data Buku ke-3

Judul Buku : Narnia
Pengarang : C.S. Lewis
Penerbit : Gramedia
Tahun Cetak : 2004
Kategori : Anak

=====


```

=== DAFTAR ISI BUKU TOKO ===
=====

Data Buku ke-1
Judul Buku      : langit
Pengarang       : tereliye
Penerbit        : gramedia
Tahun Cetak     : 2009
Kategori        : Semua Umur
-----

Data Buku ke-2
Judul Buku      : tentang kamu
Pengarang       : faiz
Penerbit        : gramedia
Tahun Cetak     : 2010
Kategori        : Dewasa
-----

Data Buku ke-3
Judul Buku      : kisah nabi
Pengarang       : balya
Penerbit        : gramedia
Tahun Cetak     : 2015
Kategori        : Semua Umur

```

2.4 Penjelasan Coding dan Hasil Output (Analisa Tugas)

1. **Penerapan Inheritance:** Class `BukuToko` dideklarasikan sebagai anak dari class `Buku` menggunakan keyword `extends`. Hal ini membuat `BukuToko` secara otomatis mewarisi atribut `judul`, `nama_pengarang`, `penerbit`, dan `tahun_cetak` tanpa harus mendeklarasikannya ulang. Hal ini mendukung konsep *code reuse* (penggunaan ulang kode).
2. **Penggunaan `super`:**
 - o Pada konstruktor `BukuToko`, keyword `super(...)` digunakan untuk meneruskan nilai atribut yang diwarisi ke konstruktor class induk (`Buku`).
 - o Pada method `displayInfo()` yang di-*override*, `super.displayInfo()` dipanggil agar method tidak perlu menulis ulang kode `System.out.println` untuk judul, pengarang, penerbit, dan tahun. Method anak hanya perlu menambahkan baris kode untuk menampilkan `Kategori`.
3. **Hak Akses (Access Modifier):** Atribut pada class `Buku` menggunakan modifier `protected`. Ini memungkinkan class `BukuToko` untuk mengakses atribut tersebut

secara langsung jika diperlukan, namun tetap terlindungi dari class di luar package yang bukan merupakan turunannya.

4. **Logika Kategori:** Method `getNamaKategori()` menggunakan struktur kontrol `switch-case` untuk mengonversi kode input pengguna (`su`, `r`, `d`, `a`) menjadi teks deskriptif (`Semua Umur`, `Remaja`, `Dewasa`, `Anak`) agar output yang ditampilkan kepada pengguna lebih mudah dibaca (*user-friendly*).
 5. **Polimorfisme & Array Objek:** Pada class `MainTokoBuku`, array `daftarBuku` bertipe `BukuToko` digunakan untuk menampung 3 objek. Pemanggilan method `daftarBuku[i].displayInfo()` di dalam perulangan `for` adalah bukti dari konsep polimorfisme, di mana sistem secara otomatis mengeksekusi method `displayInfo()` versi `BukuToko` (yang sudah di-*override*), bukan versi `Buku`.
-